An Efficient Context Management System for On-Device LLMaaS

Wangsong Yin*
Peking University
Beijing, China
yws@stu.pku.edu.cn

Mengwei Xu†
BUPT
Beijing, China
mwx@bupt.edu.cn

Yuanchun Li
Institute for AI Industry Research (AIR), Tsinghua University
Beijing, China
liyuanchun@air.tsinghua.edu.cn

Xuanzhe Liu†
Peking University
Beijing, China
liuxuanzhe@pku.edu.cn

Abstract

Large language models (LLMs) are renovating the mobile AI, catalyzing novel mobile applications such as UI task automation. A new paradigm of mobile AI ecosystem emerges in the LLM era: LLM as a mobile OS service (LLMaaS), where LLM runs as a system service and exposes its functionality (language understanding and generation) to third-party apps. As a giant step towards on-device LLMaaS, this work presents `Libra`, a system that tackles with challenge in managing persistent LLM contexts (KV cache) under tight memory constraint. `Libra` manages the LLM contexts based on the fine-grained, chunk-wise, globally-optimized KV cache compression and swapping. Specifically, it integrates three novel techniques: (1) Tolerance-Aware Compression applies different compression rates to each chunk based on their attention scores. (2) Swapping-Recompute Pipeline simultaneously swaps and recomputes the LLM contexts to improve the hardware resource utilization. (3) Chunk Lifecycle Management judiciously determines when and what to evict to reduce the context switching overhead. Through comprehensive experiments on various edge devices, `Libra` reduces the context switching latency by up to 20× and on average 9.7× compared to competitive baselines.

CCS Concepts

• **Computing methodologies** → *Natural language processing*; *Natural language generation*; • **Computer systems organization** → *Embedded software.*

Keywords

LLM as a Service, KV Cache Management

ACM Reference Format:
Wangsong Yin, Mengwei Xu, Yuanchun Li, and Xuanzhe Liu. 2026. An Efficient Context Management System for On-Device LLMaaS. In *ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems (SenSys '26), May 11–14, 2026, Saint Malo, France.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3774906.3800479

*Wangsong Yin is affiliated with School of Computer Science and Key Laboratory of High Confidence Software Technologies (Ministry of Education).
†Corresponding author.

SenSys '26, Saint Malo, France

ACM ISBN 979-8-4007-2309-4/26/05
https://doi.org/10.1145/3774906.3800479

1 Introduction

The recent progress of Large Language Models (LLMs) is reshaping the mobile AI landscape. LLMs can comprehend human language and handle most (if not all) language-based ML tasks with a huge knowledge base, e.g., language translation [26, 75], Q&A [33, 66], and smart reply [3]. More importantly, it catalyzes novel mobile applications such as UI automation tasks based on user instructions, e.g., "forward the recent 5 emails from Bob to Alice" [78]. In a nutshell, LLM marks a giant step for mobile devices towards more intelligent and personalized assistive agent [49].

Being more powerful and intrusive into user-device interactions, LLMs are eager for on-device execution to better preserve user privacy. For instance, mobile UI automation task takes in the screen information (e.g., view hierarchy [76, 78]), which can contain highly privacy-sensitive information like chatting history, photos, and textual input. Beyond privacy benefits, on-device LLM also alleviates the high resource intensity of datacenters and guarantees the availability of LLM service with limited network.

Indeed, there have been tremendous efforts made towards on-device LLM in past few years. On the one hand, more compact and resource-efficient LLMs (e.g., Gemma-2B and Falcon-1B [24, 36]) are released, and various compression algorithms (e.g., 4-bit or even 1-bit quantization [35, 53]) are proposed to effectively reduce the resource consumption of LLMs. On the other hand, system- and hardware-level support for LLMs are maturing [10, 70, 87, 90]. For instance, Qualcomm Snapdragon 8gen3's NPU is capable of executing LLM at 20 tokens/second [19]. Google is also exploring built-in LLM into their off-the-shelf devices [9].

LLM-as-a-Service (LLMaaS). A new paradigm of mobile AI is *LLM as a system service on mobile devices (LLMaaS).* It indicates that the mobile OS exposes an LLM and its inference infrastructure as a system feature to mobile apps, akin to the location or notification services. The interface between apps and `LLM Service` is based on prompts in nature language. This paradigm fundamentally differs from prior arts that apps own their models separately, into which the OS has no visibility. Such a paradigm shift is natural in LLM era, motivated by following observations: (1) LLM has world knowledge and can support generic ML tasks [22, 28, 42, 74] through properly curated or even learned prompts. (2) LLMaaS needs only one copy of LLM weights in memory, regardless of how intensively the LLM is used across apps; otherwise, the LLMs owned by different apps easily blow up the device memory; (3) A system-level LLM

can be better customized for on-device accelerator and enjoy the performance gain over commodity hardware. An exemplification of LLMaaS paradigm is the recently released Android AICore [1], a standalone LLM system service that is already in use by several Google apps for on-device summarization and Gboard smart reply.

To fulfill the vision of LLMaaS, this work identifies and tackles a unique system challenge: *LLM context management.* Specifically, unlike traditional DNNs that execute in a stateless manner, LLMs execution often needs to maintain persistent states (mainly KV cache [64]) across multiple invocations. For example, a smart reply app [3] needs to remember its historical conversation text to generate more accurate reply suggestions. According to our preliminary experiments in §2, a single LLM context could consume significant device memory (e.g., 2GB for Llama2-7B with 4k context window size); more of such LLM contexts soon dominate the memory usage of `LLM Service`. Consequently, how to properly manage the persistent LLM contexts across apps becomes crucial to improve the quality-of-service for LLM. One might treat LLM context memory as part of the app memory and reuse the mobile OSes' memory manager (e.g., low memory killer [2]) to manage them altogether. This approach, however, is inefficient due to the unique characteristics of LLM contexts (details in §2.2).

`Libra` This work presents a first-of-its-kind system for LLMaaS, named `Libra`, that decouples the memory management of LLM contexts from mobile apps. `Libra` aims to minimize the LLM context switching overhead under a tight memory budget, akin to the traditional mobile memory mechanisms that focus on reducing the app cold-start latency [25, 63, 92]. To alleviate the limitation of device local memory, `Libra` introduces novel techniques on fine-grained, chunk-wise, globally-optimized KV cache compression and swapping. `Libra` splits KV cache into a series of flexible chunks — memory blocks covering the same number of tokens (all layers in one token). Each chunk is compressed and swapped out/in independently. Similar to the OS's paging mechanism, we observe that the idea of chunking strikes a good balance between the utilization of device memory and I/O bandwidth, and outperforms token-level or context-level management.

Chunking fully leverages the unique characteristics of KV cache for optimizing context switching. (1) KV cache is easy to be chunked. As shown in §3.1, its layout grows at the token dimension that can be divided into chunks with flexible granularity. (2) KV cache is unevenly tolerant to compression. A portion of KV cache can be compressed more aggressively. Compressing in chunk level allows for maximizing the compression potential of KV cache. (3) KV cache can be recomputed. As an intermediate activation of LLM, KV cache can be recomputed from the prompt text to recover it into memory. By managing KV cache in chunks, during context switching, chunks that need to be loaded from disk can be concurrently recovered into memory through both recompute and I/O, thereby fully utilizing hardware.

`Libra` fully explores the design space of chunk-level memory management, and incorporates three novel techniques.

(1) Tolerance-Aware Compression (§3.2). `Libra` uses the accumulated attention scores of a chunk as a metric to quantify its tolerance to compression. Attention scores indicate the level of attention that the tokens pay to each other. The rationale is that a token that attracts more "attention" from others is more likely to be important. `Libra` then judiciously determines the compression rate for each chunk to maximize the overall attention scores of a context, while meeting a global average compression ratio configured by the OS.

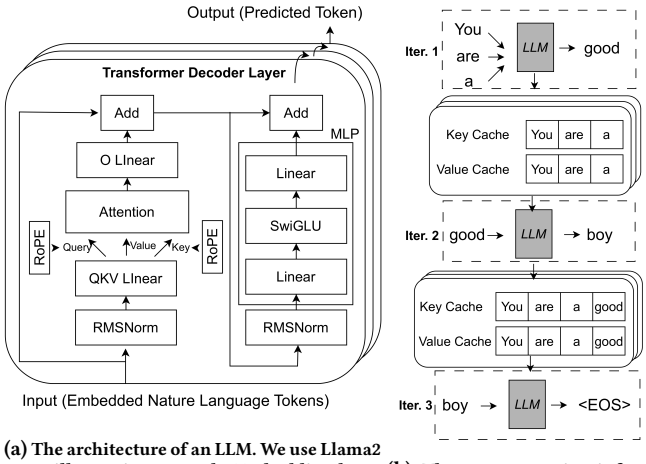
(2) Swapping-Recompute Pipeline (§3.3). `Libra` selectively recomputes arbitrary interleaved chunks from their original text and overlaps it with I/O time of other chunks. Such a pipeline is tailored for `Libra`'s chunked context memory model, realizing a much more fine-grained recompute-I/O management at the level of individual chunks. It (i) reduces the context loading latency by elastically balancing I/O and recompute after applying tolerance-aware compression; (ii) offers a design philosophy for chunk lifecycle management, specifically the chunk eviction policy that swaps heavy chunks first to maximize the pipeline performance.

(3) Chunk Lifecycle Management (§3.4). `Libra` designs the lifecycle of KV cache chunks to be more friendly to context switching. Regarding which chunk to swap out, it employs an LCTRU (Least Compression-Tolerable and Recently Used) queue to determine the eviction priority. Its rationale is that swapping heavy and least recently used chunks out to disk can better leverage chunks' time locality and `Libra`'s swapping-recompute pipeline. Regarding when to swap out, it adopts an ahead-of-time swapping-out approach to hide the time for reclaiming memory during context switching.

Results We have fully implemented `Libra` on top of Commercial Off-The-Shelf (COTS) devices including Jetson Orin NX [7], Jetson TX2 [8], and MI14 smartphone [15] with Llama2-7B [74] and OPT-7B [98]. We then evaluate its performance with a 72-hour-long context switching trace synthesized from 6 representative NLP datasets. The results show that `Libra` significantly outperforms its baselines. It reduces switching latency by up to 2 orders of magnitude compared to managing contexts via the conventional app-level memory manager low-memory killer [2], or directly managing contexts via disk swapping. Compared to the state-of-the-art chunk-based context managing system vLLM [46] with statically quantizing all chunks to 8-bits [84], `Libra` achieves up to 20× and on average 9.7× switching latency reduction. `Libra` achieves the aforementioned latency reduction without any noticeable accuracy loss on 8 datasets/benchmarks. For the first time, `Libra` addresses the issue of LLM context switching, enabling LLMaaS to provide low switching-latency and stateful services for mobile apps.

Contributions This work makes following contributions.

- We paint a picture of LLM as a system service (LLMaaS) to fully leash the power of LLM on devices, presenting its strong motivations and the key challenge of LLM context management.
- We present `Libra`, a context management system for on-device LLMaaS based on fine-grained, chunk-wise KV cache compression and swapping. `Libra` aims to maximize the context switching speed through a set of novel techniques, including tolerance-aware compression, swapping-recompute pipeline, and strategic chunk lifecycle management.
- We prototype `Libra` and comprehensively evaluate its performance on COTS mobile devices and typical LLMs. The results demonstrate the efficacy of `Libra` compared to competitive baselines.



(a) The architecture of an LLM. We use Llama2 as an illustrative example. Embedding layer is omitted for simplicity.

(b) The autoregressive inference with KV cache.

Figure 1: Illustrations for LLM's representative architecture and inference procedure.

2 LLMaaS: Motivations and Challenges

2.1 On-Device LLM as a Mobile OS Service

Large language models. Large Language Models (LLMs), such as GPT4 [22], Llama2 [74], Gemini [72], etc., are transforming mobile AI. Many cutting-edge applications are empowered by LLM, encompassing agent-based UI automation [77, 80], app built-in chatbot [13], smart voice assistant [11], automated email writer [5], etc.

We briefly introduce the model architecture and inference procedure of LLMs. The main body of LLMs is a series of stacked transformer [75] decoder layers shown in Figure 1a, where input tokens (natural language word pieces) are processed and new output tokens are predicted. Its key component is the attention mechanism, where tokens are mapped to Query, Key and Value tensors to compute the cross-token connections. LLMs perform inference in an autoregressive manner: in each iteration, it predicts a new token by history tokens (i.e., prompted tokens and generated tokens). Specifically, LLM caches the Key and Value tensor of previous tokens, known as `KV Cache` [64], to avoid repeated computations. We show such an inference procedure in Figure 1b. In iteration 1, the LLM is prompted by "You are a". The model predicts a new token "good", and saves the KV cache "You are a". In iteration 2, a new token "boy" is jointly predicted by input token 'good" and KV cache "You are a". Meanwhile, the KV cache "good" is also saved. In iteration 3, an "<EOS>" token is predicted by input token 'boy" and KV cache "You are a good", and the LLM inference ends.

On-device LLM. A growing demand is to deploy LLMs on devices to support privacy-preserving mobile AI. Taking agent-based UI automation as an example, it takes screen UI as input [78], which is extremely privacy-sensitive as it might include chat history, photos and account information rendered during UI operation. On-device LLM alleviates such privacy concerns as no data leaves devices. Moreover, on-device LLM improves service availability and cost efficiency without relying on network and expensive cloud GPUs.

Recently, remarkable progress has been made towards the fast and energy-efficient on-device inference of LLMs, encompassing a full-stack optimization from algorithm to hardware [10, 19, 70, 87, 90]. For example, mobile SoC vendors have already provided off-the-shelf hardware support for LLMs. For instance, Qualcomm reports Snapdragon 8gen3's NPU (an ASIC specialized for DNN inference) to be "meticulously designed with generative AI in mind", capable of executing LLM at 20 tokens/second [19].

On-device LLM as a mobile OS service. Currently, there is a trend of major paradigm shift of mobile AI when on-device LLM matures: *LLM as a system service on mobile devices (LLMaaS).* It indicates that, the mobile OS exposes an LLM (as well as the inference infrastructure) as a system feature to apps for use, just like location and notification services, instead of each app owning an LLM individually. This shift is supported by a recent survey on mobile agent where many industry experts explicitly call for OS-level LLM [49]. Meanwhile, Google has recently released a preview of such LLMaaS design, named AICore, as an Android system service [1]. The service is already in use by several Google products, such as voice recorder and smart reply. The interface between apps and `LLM Service` is text in nature language: app sends prompts to `LLM Service` as LLM inputs; `LLM Service` sends back tokens generated during LLM autoregressive inference. §3.1 will show a concrete API design of `LLM Service`.

LLMaaS is backed up by following observations.

- **LLMaaS is feasible.** LLM has world knowledge and can support generic ML tasks [22, 28, 42, 74]; in-context learning [34, 58] and PEFT [44, 56] are capable of further enhancing LLM's capability on downstream tasks. A recent study [96] shows that a 7B-sized LLM can achieve better accuracy on most mobile AI tasks compared to the non-LLM models.
- **LLMaaS is desirable.** Sharing one LLM across apps guarantees affordable, mostly static memory footprint; otherwise, the LLMs owned by each app easily blow up the device memory. The apps might load the LLMs on demand to save memory, yet the weights I/O time easily overwhelms the token generation process: loading LLaMA-7B (4-bit) takes 4.76 seconds, which equals the time to generating 95 tokens on MI 14 smartphone.
- **LLMaaS facilitates hardware and OS design.** On one hand, LLMaaS facilitates the accelerator design on mobile SoCs, since the LLM architecture becomes static and determinable by device vendors. A recent paper [96] shows that only the most common 13% of operators can be fully executed on mobile NPUs. On the other hand, LLMaaS grants the full visibility of LLM execution to OS, who thus can schedule, batch, and cache-reuse the inference requests from different apps for better energy efficiency or throughput [37, 95].

2.2 LLMaaS Context Management

A crucial system challenge this work identifies and tackles is *how to efficiently manage the LLM contexts.* When an app uses `LLM Service`, it maintains a stateful LLM context, including mainly the KV cache as aforementioned. Notably, long model contexts are important to LLM-powered mobile apps to provide customized and stateful functionality. With a long context, the mobile app can define more

complex tasks at a time and augment each task with more information so that it can generate more accurate and personalized output.

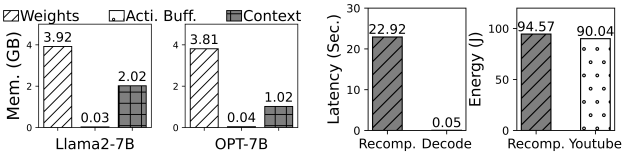
Observation#1: LLM contexts are memory-intensive. Since memory is a scarce resource in executing LLMs [23, 70, 94], we study the memory footprint of typical device-affordable LLMs and break down it into three categories: LLM weights, activation buffers, and contexts. As shown in Figure 2a, the model contexts contribute significantly to the memory footprint. For instance, a single context with the maximal context window (4k tokens) of Llama2-7b consumes over 2GB memory, which is over 50% of model weights. Note that the LLM weights are shared across all LLM invocations (thereby static), yet the memory usage of LLM contexts proportionally scales with the number of active contexts and longer context window (e.g., up to 128k for recent LLMs [6]). Consequently, the LLM context is more likely to be the bottleneck in `LLM Service` and needs to be carefully managed.

Observation#2: LLM contexts often need to be persistent. In this work, we use the term "persistence" to indicate that an LLM context needs to be memorized across multiple invocations that could span hours or even days, regardless of whether apps being switched to background or even killed. For example, a multi-round conversation agent needs to remember the historical input/output (though with a maximal context window size) whenever it is used, akin to ChatGPT or any other web-based chatting bots. Besides, a UI automation agent needs to memorize its history operations to serve users' further interactions, such as "order a pizza in the same restaurant as yesterday". Furthermore, the smart reply of Google Gboard calls `LLM Service` to realize "generating reply suggestions based on the full context of a conversation, not just a single message." [3]. Such persistence feature fundamentally differentiates LLM from prior deep learning models like CNN whose each invocation is independent.

Without system-level persistence support in `LLM Service`, the apps could instead remember the LLM input/output and feeding them to LLM to "replay" (or recompute) the whole LLM context at each invocation. This approach, however, not only incurs extra programming efforts to developers, but are also highly inefficient on mobile devices. We conducted preliminary experiments to showcase the resource cost of such context recomputing. As shown in Figure 2b, recomputing a Llama2-7B's context takes 22.92 seconds on MI14, while decoding a token for users only takes 0.05 seconds. The energy consumption of one recompute is roughly equivalent to watching one minute of YouTube video.

Observation#3: the conventional app-level memory management is not satisfactory. One intuitive and plausible design is to account the LLM context memory as part of the whole app's memory who actually uses the LLMaaS, and rely on the app memory manager to manage them as a whole. For example, mobile devices typically employ low-memory killer (LMK) [2] that kills background apps and frees their memory when system goes out of physical memory. In this approach, the app memory and LLMaaS context memory are managed together without differentiating them, e.g., both get released if either of them is too large when out of memory.

However, we find this approach not efficient, since the context memory and app memory are inherently different in following aspects. (1) LLM contexts are more expensive in terms of time/energy expenditure to obtain. Constructing a context memory takes much time and energy, as this procedure is via LLM inference. As shown in Figure 2b, killing an app and recomputing its KV cache incur huge time and energy overhead (e.g., 22.92 seconds and 94.57 J). (2) LLM contexts are relatively cold, e.g., used in low frequency, even when LLM becomes an indispensable feature on mobile devices. For instance, Gboard only invokes `LLM Service` for smart reply functionality during chatting on telegram. Consequently, a lightweight app could easily get killed by OS's LMK if it possess an active LLM context, which though will not be used in a near future. (3) LLM contexts are naturally compressible, as will be discussed in §3.1, while app memory mostly not. Treating the memory of LLM contexts and app as a whole misses such optimization opportunity. Instead, we propose decoupling the management of app and model context memory for better LLM efficiency.



(a) Memory breakdown. **(b) Recompute overhead.**

Figure 2: Experiments results on MI14. Experiments in (a) is performed on Llama.cpp framework [10]. "recomp." in (b) means recompute; "Decode" means generating a token; "Youtube" means watching videos on YouTube for 1 minute.

Interface	Descriptions
Class `LLMService`	A system service class that is similar to the Android's `android.app.Service` class.
Class `LLMCtx`	A class that defines LLM context. `LLMService` interacts with apps via `LLMCtx`.
Method `newLLMCtx(` *Optional* `systemPrompt` `)->LLMCtxStub`	A method that returns a stub of a new LLM context. On initializing, optional system prompts can be assigned to `LLMCtx`.
Method `bindLLMService(` `app` `)->LLMService`	A method that binds the `LLMService` to an app.
Method `callLLM(` `LLMCtxStub, newPrompt` `)->outputs`	A method that calls `LLMService` via an `LLMCtx`. It takes an `LLMCtxStub` and a new prompt as input, and returns the updated `LLMCtxStub` and decoding results.
Method `delLLMCtx(` `LLMCtxStub` `)`	A method that deletes an `LLMCtxStub`.

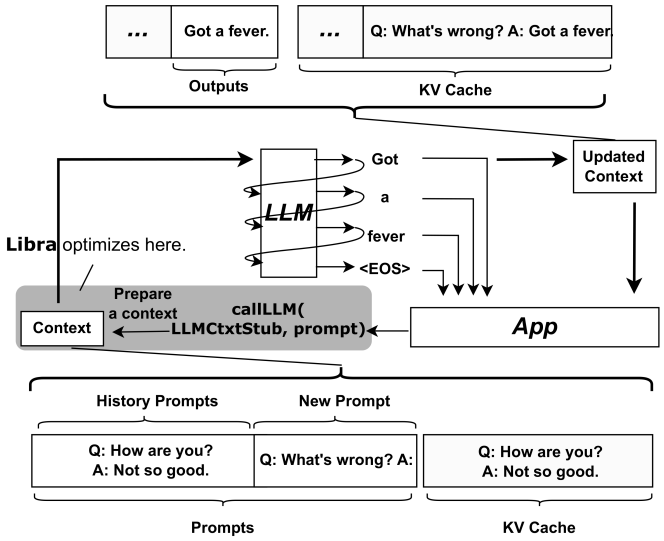
Table 1: On-device LLMaaS interface abstraction.

3 Libra Design

3.1 Libra Overview

This work advances on-device LLMaaS with `Libra`, a first-of-its-kind `LLM Service` design on devices with a dedicated memory management mechanism of LLM contexts.

LLMaaS APIs. As shown in Table 1, We abstract LLMaaS interfaces to be compatible with Android Services. Apps interact with `LLMService` via `LLMCtx`. Specifically, we show a chatbot app in




```
1  public class ChatBot extends AppCompatActivity
       {
2    /* App codes */
3    ...
4    private void newSession(){
5      // bind LLM service to current app
6      LLMService llmService =
7        bindLLMService(curApp);
8      // new LLM context
9      LLMCtxStub llmCtx = newLLMCtx();
10     // chatbot logic
11     while(prompt = getText()){
12       String result =
13         llmService.callLLM(llmCtx, prompt);
14       Log.d(result);
15     }
16     // delete LLM context when session ends
17     delLLMCtx(llmCtx);
18   }}
```

Figure 3: Left: a workflow of a chatbot app calling `LLM Service` via `Libra`. Right: pseudo codes in Java.

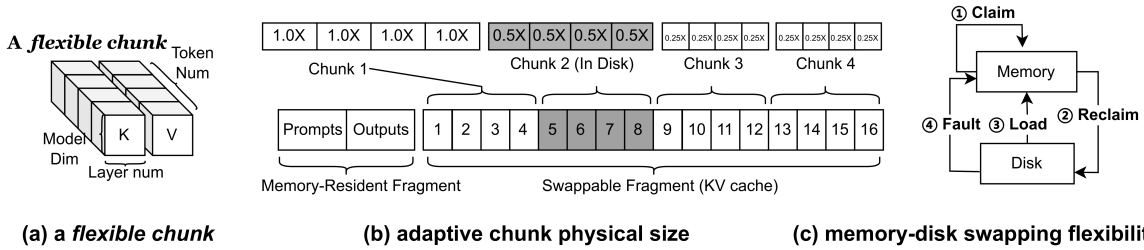


Figure 4: `Libra`'s chunk-wise memory model. Its flexible chunks are memory blocks that contain same number of tokens. Each flexible chunk can be adaptively compressed and can be independently swapped to disk.

Figure 3. Each round, an app appends new prompts to `LLMCtx` and invokes `LLMService`. When token generation completes, `LLMCtx` is updated. Such a chatbot can hold multiple contexts, which are persistent until the app explicitly deletes them through `delLLMCtx()`. `Libra` globally configures the maximal context numbers per app and the maximal context length. `Libra` allows each app to hold up to `K` (a configurable system parameter) active LLM contexts. Note that each active LLM context has a maximal memory usage with a maximal context window size supported by the LLM.

`Libra`'s design goal. `Libra` optimizes for the Quality-of-Service (QoS) of the LLMaaS. Specifically, `Libra`'s design centers around the memory inefficiency challenge as presented in §2.2, aiming to minimize the *context switching latency*, akin to how the mobile memory managers minimize the app cold start latency [25, 63, 92]. For example, in Figure 3, `Libra` ensures fast context preparation (i.e., make it in memory for LLM inference) when a context is invoked. This goal has not been explored in prior literature, and is fundamentally different with (as well as orthogonal to) LLM inference speed optimizations during continuous token generation [70, 87]. As elaborated in the following, `Libra` realizes the design goal with a dedicated memory management system that is transparent to (1) LLMaaS callers (2) app developers (3) the LLM inference engine.

`Libra`'s context management system. Illustrated in Figure 4, `Libra` manages the context memory in flexible chunks. As show in Figure 4, a flexible chunk is a KV cache sub-tensor with the same shape (i.e., model dimension, layer number and token number, globally configured by the OS at installation time). By carefully selecting the chunk size (token number), `Libra` can utilize device RAM with minimal fragmentation compared to managing the context as a whole. Such a chunk size is set empirically to a default number 16 tokens on mainstream mobile SoCs. We demonstrate its effectiveness and discuss its selection rationale with experiments in §5.4.

A flexible chunk is further designed with the following advanced features.

• *Adaptive physical size.* Each chunk is allocated with an adjustable physical size based on its semantic representation to the LLM inference. As shown in Figure 4b, apart from a limited fragment of prompt/output texts that can be effortlessly held in memory, the context main body, KV cache, is split to chunks with various compression ratios. The rationale behind it is that (1) unlike objects,

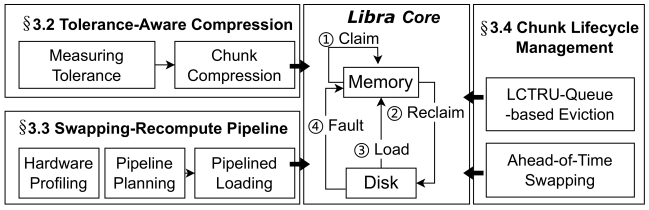


Figure 5: Main modules of Libra's memory management system.

KV tensors can be approximately compressed (2) as a set of tokens, a chunk has its semantic representations to the LLM. With an adaptive physical size, the limited device RAM and disk I/O can be significantly better utilized.

• *RAM-disk swapping flexibility.* LLMaaS context memory is relatively cold compared to app memory. Swapping is thereby introduced to chunk management. As is shown in Figure 4b/c, each chunk can be swapped out to disk independently. The swapping logic is characterized by a few primitives defined by Libra. ①Claim, which directly allocates free memory to a chunk; ②Reclaim, which swaps a chunk out to disk and reallocates its memory to a new chunk when memory is under pressure; ③Load, which moves a missing chunk from disk to memory before LLM inference; ④Fault, which moves a missing chunk from disk to memory at each LLM inference iteration.

In the rest of the section, we introduce the three key techniques of Libra to meet its design goal. They are depicted in Fig 5: (1) Tolerance-Aware Chunk Compression (§3.2) adopts chunk-wise compression to minimize I/O overhead without noticeable accuracy loss; (2) Bubble-Free Pipelined Chunk Recovery (§3.3) further utilizes the idle computing to accelerate context switching; (3) Full Lifecycle Chunk Management (§3.4) judiciously decides which chunk and when to swap-out to enhance Libra's QoS.

3.2 Tolerance-Aware Compression

Intuitively, chunks should exhibit various levels of compression tolerance, as they contain various semantic information.

Measuring the tolerance. We define compression tolerance D_i of the i_{th} chunk by its accumulated attention scores:

$$D_i = \frac{1}{q-p}\sum_{col=p}^{q}\frac{1}{L}\sum_{l=0}^{L}\frac{1}{H}\sum_{h=0}^{H}\frac{1}{R-row}\sum_{row=0}^{R}A^{l,h}_{row,col}, \quad (1)$$

where $A^{l,h}_{row,col}$ is the attention score of the l_{th} layer and h_{th} head, and the i_{th} chunk contains tokens from the p_{th} to q_{th}. Specifically, as shown in Figure 6a, the attention score is a R rows and C columns lower triangular matrix calculated by $A = softmax(mask(\frac{Q\cdot K^T}{\sqrt{d_k}}))$ [75]. Each row represents the "attention score" that a token pays to others. The scores are softmaxed and sums to 1.0 at each row. For instance, the number "0.5" at "a" row and "are" col means that the token "a" pays 50% attention to "are". If a token is always paid more attention by other tokens, it should be more informative. We conduct pilot measurement on Wikitext-2 [57] using Llama2-7B [74]. Figure 6b shows that most (over 80%) tokens attract relatively low attention. Figure 6c shows that such scores are stable across layers. Thereby, Libra takes the average of a token's column in attention score matrix, and further accumulate it by heads, layers and tokens to achieve chunk-level D_i, as shown in Equation 1.

Compressing chunks. As tokens exhibit various levels of compression tolerance, Libra provides multiple levels of compression ratio for chunks, denoted as $\{ratio_w\}$. Before compression, Libra computes each chunk's D_i and determines its ranking $Rank_i(\%)$ among all other chunks in the context. Then a series of thresholds $\{\sigma_{ratio}\}$ are formed. Chunk i is compressed to $ratio_w$ based on $Rank_i$, *s.t.*,

$$\sigma_{ratio_{w+1}} < Rank_i \leq \sigma_{ratio_w}. \quad (2)$$

The thresholds are formed by maximizing the overall accumulated attention scores of a context under a given global average compression ratio $ratio_{global}$. Libra maximizes

$$ctxD = \sum_w ratio_w \sum_{\sigma_{ratio_{w+1}} < Rank_i \leq \sigma_{ratio_w}} D_i,$$
$$s.t., \sum_w ratio_w \cdot (\sigma_{ratio_w} - \sigma_{ratio_{w+1}}) = ratio_{global}. \quad (3)$$

Formula 3 is solved by a lightweight grid search, which incurs negligible overhead (e.g. <1ms) on devices.

In practice, Libra adopts quantization for compression. A chunk can be quantized to 8bits, 4bits, or 2bits. Its detailed configurations are elaborated in §4. To be noted, as a system service, Libra does not differentiate the type of context owner, maximizing fairness akin to traditional OS scheduling policies like Max-Min Fairness [65]. As a result, Libra sets a global compression ratio $ratio_{global}$ to *each LLMaaS context*. Such a value is configurable by the OS to achieve a trade-off between QoS and resource. By default, Libra empirically sets $ratio_{global}$ to a maximal number that is lossless on all downstream tasks (configured in §4 and discussed in §5.2). The global LLMaaS context memory management across apps does not require the context memory being possessed by apps. Instead, the memory space is actually disaggregated from apps and maintained by the LLMaaS process, which is very convenient for global scheduling.

3.3 Swapping-Recompute Pipeline

The Load primitive in Libra loads missing chunks from disk to memory upon calling callLLM(). Libra introduces recompute to swapping-in to further accelerate it, as processor is idle during disk I/O. Recompute here means that use the original prompt text pieces to calculate a portion of KV cache, as KV cache is essentially LLM activations. By recomputing some chunks in a pipelined manner instead of loading them through disk I/O, we can fully utilize the hardware.

Making interleaved chunks recomputable. Recall that in Libra's context memory model, each chunk can be swapped out to disk. Thereby, Libra faces a challenge: employing recompute to recover interleaved non-contiguous chunks. To do so, Libra refines the LLM recompute procedure. Given a chunk-wise KV cache $\mathbf{C} = \{chunk_i\}$ and corresponding prompt text $\mathcal{T} = \{text_i\}$, Libra recovers the missing chunks $\{chunk_o\}$ through $\{text_o\}$ and $\{chunk_i\} - \{chunk_o\}$. Specifically, Figure 7 gives an example of Libra's chunk-recomputing procedure. In Figure 7, the KV cache

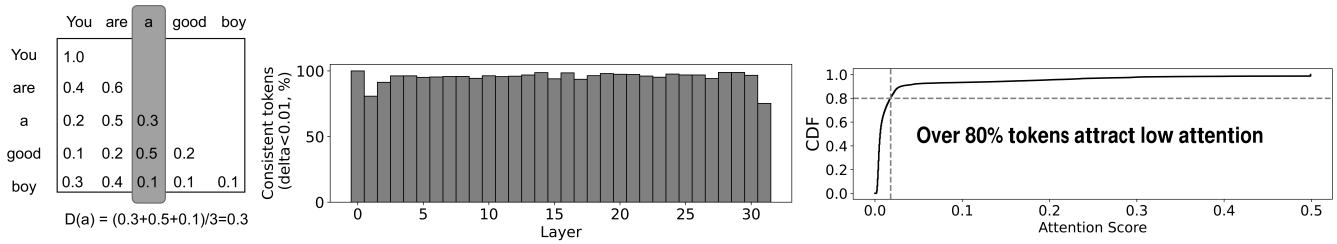


Figure 6: Libra's tolerance-aware compression. (a) Calculating attention scores for token "a". (b) Distribution across layers. (c) Distribution across tokens.

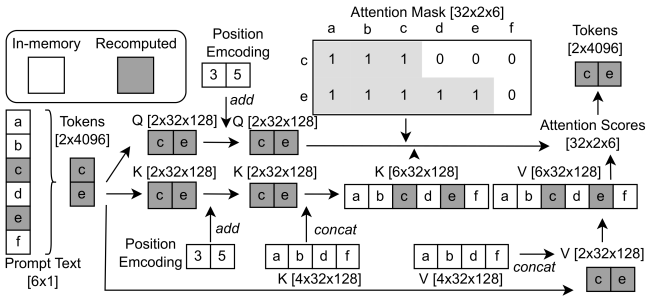


Figure 7: An example of Libra's chunk-recomputing procedure. We use a Llama2-7B layer here for representation. It has 32 heads and 4096 hidden size. Libra recomputes the missing "c" and "e" in KV cache based on their prompt text and other tokens' KV cache.

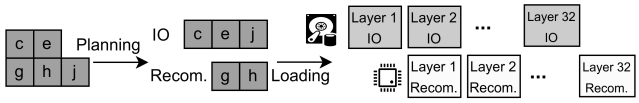


Figure 8: Swapping-recompute pipeline.

of a text sequence "a b c d e f" is partly swapped out of memory ("c" and "e"). To recover, Libra embeds "c" and "e" to tokens, and recomputes them to get Q, K and V tensors. Here, Libra applies a global position encoding to Q and K, i.e., encoding "c" with position 3 and "e" with position 5. Then, the recomputed K/V of "c" and "e" is inserted to the K/V of "a", "b", "d" and "f"; the entire K/V is recovered in current layer. After that, Q is calculated with the recovered K to get the attention scores. The attention mask is a causal mask [75] that retains all tokens before the current token. In Figure 7, the attention mask masks out "d e f" for "c" and "f" for "e". Finally, the tokens that input to the next layer are calculated by the attention scores and the recovered V. In doing so, Libra realizes the exact recompute of any chunks.

Swapping-recompute pipeline. Libra concurrently recomputes a portion of chunks and swaps other chunks into memory from disk.

As shown in Figure 8, I/O and recompute are overlapped in a pipeline, where the next layer's I/O is performed during the current layer's recompute.

Such a pipeline is elastic: Libra can adjust the loading configuration (i.e., which chunks are recomputed and which chunks are swapped) to maximize the efficiency. Libra plans the configuration of each loading based on offline profiled hardware information.

i. Profiling. Recompute delay $T_{re}(x, f, e)$ is a function w.r.t. the number of chunks x, the hardware frequency f and the energy mode e. IO delay $T_{IO}(m)$ is a function w.r.t. the size m of on-loading chunks. In practice, we approximate T_{re} and T_{IO} with linear functions. Libra performs a cost-effective one-shot measurement with discrete test points at installation time. The above functions fit these test points.

ii. Planning. Libra plans its elastic pipeline by solving the following optimization. Given the on-loading memory size m and the number of chunks $\{x_{ratio_w}\}$ with different compression ratios, Libra minimizes the following equation by determining a proper number of recomputed chunks $\{x^{re}_{ratio_w}\}$ with different compression ratios.

$$\begin{aligned} pipelineDelay = max[T_{re}(\sum_w x^{re}_{ratio_w}), \\ T_{IO}(m - \sum_w ratio_w \cdot x^{re}_{ratio_w})], \\ s.t., \forall w, x^{re}_{ratio_w} < x_{ratio_w}. \end{aligned} \tag{4}$$

Such a problem is solved by a linear programming, which incurs negligible overhead on devices.

Generalize the pipeline across hardware. The characterization of recompute latency function $T_{re}(\cdot)$ and IO latency function $T_{IO}(\cdot)$ is general across mainstream mobile processors, encompassing CPU, GPU and NPU that collaborate with tiered memory. As a result, Libra only needs to perform a one-shot, lightweight hardware-specific profiling at installation time for each envelope of compute ability and memory bandwidth. No extra modification of design is required for generalization. We have demonstrated the generalization on a variety of hardware in §5.

3.4 Chunk Lifecycle Management

Libra manages the lifecycle of KV cache chunks to be more friendly to context switching. It mainly decides which chunk and when to swap out. Regarding which chunk to swap out, it employs an

Device	RAM	Disk	SoC
Jetson Orin NX	8GB	NVMe SSD	6-core Arm® v8.2 CPU 1024-core Ampere™ GPU
Jetson TX2	8GB	SATA HDD	Dual-Core Denver 2 CPU Quad-Core ARM® MPCore 256-core Pascal™ GPU
MI14 Smartphone	8GB	UFS 4.0	8 CPU cores (X4/A720/A520) Adreno™ GPU Hexagon™ NPU

Table 2: Details of mobile/edge devices we use. "RAM" stands for the default RAM volume for LLM Service.

Task	Dataset	Delta length
News Classification	AGnews [99]	0.1k–0.5k
Document Summary	Xsum [60]	1k–2k
Chat History Summary	Samsum [38]	0.1k–0.3k
Text Comprehension	cnn dailymail [43]	0.5k–1k
Translation	WMT17-de-en [27]	0.05k–0.1k
Sentiment Classification	SST-2 [69]	0.01k–0.1k

Table 3: Datasets we use for trace synthesis. An entry in a dataset is regarded as one LLM calling. "Delta length" refers to the length of context growth after each `callLLM()`.

LCTRU queue to determine the eviction priority. Regarding when to swap out, it adopts an ahead-of-time swapping-out approach to hide the time for reclaiming memory during context switching.

i. AoT Swapping. Compared to complex memory modification timing of apps [40, 47, 104], `LLM Service`'s memory modification is much easier to monitor: chunks are sequentially modified during LLM inference. Thus, swapping-out can be performed ahead of reclaim. `Libra` swaps out all the modified chunks at the returning stage of `callLLM()` (even when not under memory pressure). Its delay is imperceptible to the `LLM Service` caller. In doing so, the reclaim primitive at context preparation stage is overhead-free.

ii. LCTRU-Queue-based Eviction. A good eviction policy can reduce the overhead of swapping. App memory managing methods, such as LMK, evict memory based on the app types (ranked by oom_adj_score). As a system service, `Libra` does not differentiate the type of context owner. Its eviction policy is based on two principles: *i)* leveraging contexts' time locality; *ii)* heavy chunks should be evicted first. Principle *ii* is derived by the swapping-recompute pipeline in §3.3. Given memory size m, the total number of chunks is inversely proportional to the number of less-compressed chunks. According to Equation 4, with same memory size, fewer chunks will result in a lower pipeline delay, as the recomputing delay $T_{re}(x, f, e)$ is irrelevant to memory size.

Based on the above principles, `Libra`'s employs an LCTRU (Least Compression-Tolerable and Recently-Used) queue. The LCTRU queue consists of multiple concatenated sub-queues with different compression ratios, i.e., $Q_{LCTRU} = \{Q_{ratio_w}\}$. Each sub-queue is ordered by the recently accessed time of its in-memory chunks. Q_{LCTRU} is updated upon each invocation of `callLLM()`. When memory reclaiming occurs, `Libra` pops out the corresponding number of elements from Q_{LCTRU} based on the required memory size.

Besides, `Libra` manages a context's chunks as a memory-resident working set: during `callLLM()`, `Libra` *locks* the context memory, forbidding reclaiming their own chunks. In doing so, `Libra` avoids system thrashing and realizes being transparent to LLM inference. Notably, although it will not be triggered by `Libra`, the Fault primitive is still retained for robustly handling exceptions such as system crush.

4 Implementation and Methodology

`Libra` implementation. We have fully implemented a `Libra` prototype with 3.5k LoC in Python/C++. We implement an `LLM Service` on three representative COTS mobile/edge devices shown in Table 2. Jetson Orin NX/TX2 [7, 8] are high-end edge boards for autonomous robotics or cars. We build `LLM Service` atop Huggingface Transformers [82] and mllm [59]. The `LLM Service` runs as an independent process. It receives inference requests from client processes through socket IPC. Except for the client processes and the `LLM Service` process, we do not run any other non-essential application processes on device. We select two LLMs: Llama2-7B [74] and OPT-6.7B [98], with a maximal context length as 4K and 2K, respectively. [1] We apply a sliding window [85] to the context to enable LLM inference in a streaming manner.

The context memory management module of `Libra` is embedded within `LLM Service`. We use pickle [17] and pickle-in-cpp [18] to implement memory-disk swapping. Since the sub-byte data format is not natively supported by the LLM inference framework, `Libra` utilizes parallel bit-shift operations to pack the compressed data into a supported INT8 format. The swapping-recompute pipeline is implemented by multithread. We use an independent I/O thread to load chunks from disk to memory. The computation thread proceeds to the next layer only after the I/O thread for the current layer (reading next layer's KV cache) has completed.

`Libra` configuration. The chunk size is set to 16 tokens. We select three levels of chunk-wise compression ratio, i.e., $\{ratio_w\} = \{8/8, 4/8, 2/8\}$. The $ratio_{global}$ is set to 50%.

Context switching trace. To the best of our knowledge, there is no publicly available trace of LLMaaS context switching on devices. In order to comprehensively and accurately evaluate `Libra`, we synthesized a trace that formulated by

$$Trace = \{(Time_i, CtxtID_i, Prompt_i, groundTruth_i)\}, \quad (5)$$

Where $Time_i$ is the i_{th} calling time of `callLLM()` with $CtxtID_i$, and $Prompt_i$ and $groundTruth_i$ are the input and ideal output text. The prompts and groundTruths for a context are derived from a dataset in Table 3, while a dataset can derive multiple contexts. We generate calling time $Time_i$ using Poisson distribution with different calling rates. Akin to apps' switching [25, 63, 92], context switching pattern could be complex: it can be either irregular or with preference (e.g., influenced by invoke history or workloads). Recognizing that, we construct the following various patterns of context switching to simulate real-world scenarios.

- `Random`. Contexts are switched with the same probability.
- `Markov`. Context switching is determined by a first-order Markov process, which assigns higher priority to recently used contexts.
- `Gaussian`. Context switching follows a Gaussian distribution w.r.t delta length. In this pattern, a context with moderate workload is more likely to be requested by the apps.

[1] The weights are downloaded from the official repositories on huggingface website. We quantize LLM weights to 4-bit integers with GPTQ [35].

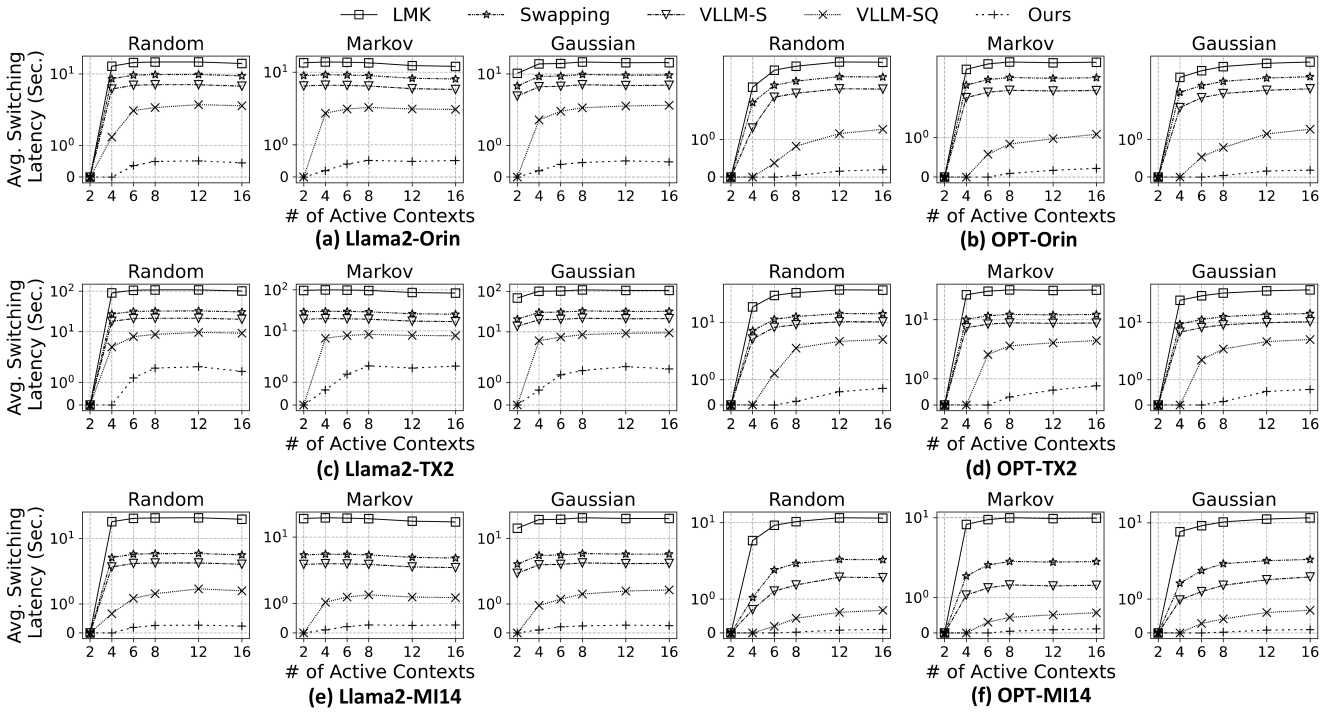


Figure 9: On-average context switching latency on a 72-hours-long trace.

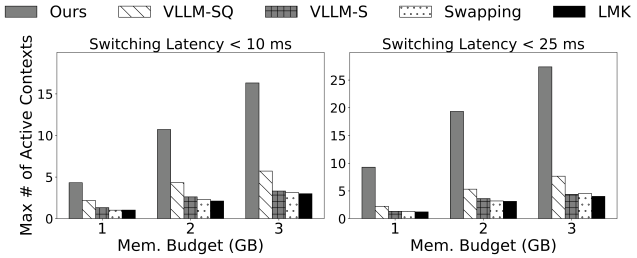


Figure 10: Performance under various memory budgets. Model: Llama2; device: Orin.

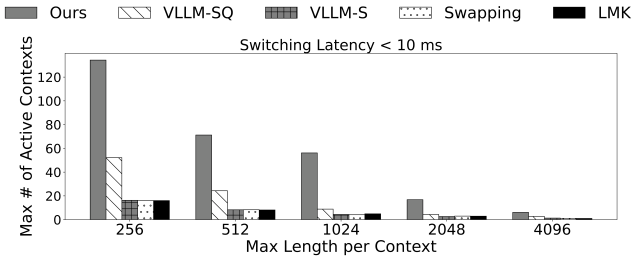


Figure 11: Performance under various maximal context lengths. Model: Llama2; device: Orin.

Note that `Libra` will not try to predict the context switching pattern, as such a pattern cannot be known as a priori.

We synthesize 72-hours-long traces with different settings to evaluate `Libra`.

Baselines. We compare `Libra` to the following alternatives by reporting the performance on the same trace:

- `LMK`. Contexts are killed by a low-memory killer [2] when memory is under pressure. The killed contexts need to be recomputed when called again.
- `Swapping`. Contexts are swapped out to disk as a whole when memory is under pressure. The swapped-out contexts need to be swapped-in when called again.
- `VLLM-S`. VLLM [46] is a state-of-the-art KV cache managing system without compression. We reproduce its chunk-wise KV cache managing on devices. Chunks are swapped under memory pressure.
- `VLLM-SQ`. KV cache chunks in `VLLM-S` are further compressed by a state-of-the-art activation quantization algorithm SmoothQuant [84]. Chunks are equally quantized to the same level (INT8).

Metrics. We mainly report two metrics of `LLM Service`'s context switching QoS: *Context Switching Latency on Average* and *Maximal Number of Active Contexts*. The former is under a given number of active contexts; the latter is under a given switching latency constraint. "Active context" refers to contexts that are persistent (have not been deleted by `delLLMCtx()` and could be invoked).

Dataset (Metric)	AGNews (Acc.)		Xsum (RG-L)		Samsum (RG-L)		CNNDM (RG-L)		WMT17 (RG-L)		SST2 (Acc.)		MMLU (Acc.)		DroidTask (Step Acc.)		Avg.	
Models/Methods	Llama2	OPT	Llama2	OPT	Llama2	OPT	Llama2	OPT	Llama2	OPT	Llama2	OPT	Llama2	OPT	Llama2	OPT	Llama2	OPT
16bits ※	**59.0**	**24.5**	**13.1**	**12.3**	**18.9**	**13.7**	**22.9**	**24.3**	**34.4**	**31.2**	**88.7**	**63.8**	**37.2**	**28.8**	**14.4**	**17.4**	***36.1***	***27.0***
Static-8 ※	**58.7**	**24.7**	**13.1**	**12.2**	**18.8**	**12.2**	**22.8**	**24.3**	**34.2**	**31.2**	**88.8**	**63.8**	**37.1**	**28.7**	**13.9**	**16.8**	***36.0***	***26.7***
Static-4	37.6	23.7	13.3	11.9	16.2	10.7	19.7	20.8	28.7	27.3	63.2	60.2	32.4	23.7	11.4	12.6	27.8	23.9
Static-2	25.3	23.2	1.1	2.7	2.2	1.7	0.9	3.7	5.9	8.7	50.0	51.0	25.1	24.2	6.7	9.8	14.7	15.6
Libra(50%) ※	**58.7**	**25.2**	**12.7**	**12.1**	**18.2**	**12.2**	**22.0**	**22.8**	**33.3**	**30.8**	**88.7**	**63.9**	**36.7**	**28.0**	**14.8**	**18.0**	***35.6***	***26.6***
Libra(35%)	54.2	23.6	10.3	11.5	16.9	11.7	19.2	19.1	28.1	29.9	83.2	58.7	31.1	24.7	8.7	14.7	31.5	24.2

Visualized average compression ratio ※ Methods with minimal quality loss (<1% on average) compared to FP16

Table 4: `Libra` achieves on-par accuracy with much better compression ratio compared to static baselines.

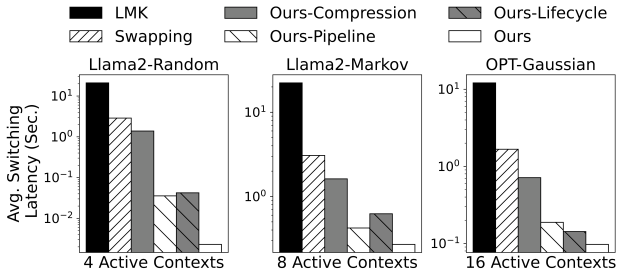


Figure 12: Ablation study.

	FP16	CG	H_2O	LG2	`Libra`	+H_2O	+LG2
MMLU Acc.	37.2	37.0	36.9	35.1	36.9	36.8	34.9
Comp. Ratio	100%	15%	40%	80%	25%	15%	20%
Overhead	N/A	47.9s	N/A	0.26s	2.1ms	2.3ms	0.29s

Table 5: Comparing to previous KV compression literature: CacheGen (CG) [52], H_2O [100], and LLMLingua2 (LG2) [62].

5 Evaluation

5.1 End-to-End Context Switching Performance

We first evaluate to what extent `Libra` enhances the context switching QoS of `LLM Service`.

End-to-end switching latency. We report the on-average switching latency of 2/4/6/8/12/16 active contexts by running the synthesized trace in §4. The maximal retained context length is set to the LLM's default context window (4k for Llama2 and 2k for OPT). The calling rate is one request in five minutes on average. All remaining memory after running the `LLM Service` on the device is dedicated to contexts. The experimental results are shown in Figure 9. The y-axis has been taken with symmetric logarithmic scale. We have the following observations.

In general, compared to the de facto app memory managing method `LMK`, `Libra` achieves a significant reduction in switching latency by up to 2 orders of magnitude; compared to the vanilla `Swapping` baseline, `Libra` reduces switching latency by 1–2 orders of magnitude; compared to other strong baselines that applies chunking+swapping (`VLLM-S`) and chunking+swapping+compression (`VLLM-SQ`) to KV cache managing, `Libra` still achieves up to 20× and on average 9.7× reduction.

`Libra` brings substantial context switching speed improvements across various context switching patterns, devices, and LLMs. We conducted experiments with identical settings for the three patterns (`Random`, `Markov` and `Gaussian`). The results indicate that `Libra` can consistently handle different context access patterns. On three different devices (Orin/TX2/MI14), `Libra` consistently demonstrates significant performance improvements. Notably, on the TX2, the overall switching latency is longer compared to the other two devices. This is due to its lower disk bandwidth (SATA HDD) and less-powerful hardware accelerators, which restrict `Libra`'s swapping and swapping-recompute pipeline. Nevertheless, `Libra` still outperforms baselines significantly, owing to its context compression and chunk lifecycle management. Additionally, `Libra` achieves substantial performance improvements across different LLMs. On the OPT model, the switching latency is lower, attributed to its smaller context window (traded for shorter memory and lower in-context learning ability).

Various memory budgets. We report the maximal number of active context under different switching latency constraints.

As shown in Figure 10, under a 10 ms latency constraint, `Libra` supports the switching between 4.32/10.72/16.32 contexts with 1GB/2GB/3GB memory budget, 1.99×/2.48×/2.85× higher than baselines; under a 25ms latency constraint, `Libra` supports the switching between 9.28/19.34/27.38 contexts with 1GB/2GB/3GB memory budget, 4.16×/3.62×/3.57× higher than baselines.

Various maximal context lengths. We evaluate maximal number of active context under various maximal context lengths. As shown in Figure 11, under a 10 ms latency constraint, `Libra` supports 2.57×/2.95×/3.31×/3.73×/2.24× more active contexts with 256–4096 maximal context lengths.

5.2 Compression Efficacy

Recall that `Libra` employs various levels of compression for different chunks. We evaluate its overall efficacy and discuss the rationale of parameter selection specified in §4. In Table 4, we compare our quantization to statically quantizing all chunks to the same bitwidth by smoothquant [84]. We mainly report zero-shot results on the datasets we choose for synthesizing our trace in Table 3 and two comrehensive benchmarks for LLM reasoning ability (MMLU [42]) and agent ability (DroidTask [79]).

Overall efficacy. We observe that our approach outperforms static methods. Aggressively compressing the entire KV cache into 4-bits/2-bits incurs significant generation quality loss (e.g., 8.2%/21.3%

on average for Llama2). In comparison, Libra's default compression strategy (row "Libra (50%)" in Table 4) achieves 2×–4× higher compression ratio with negligible quality loss.

Besides, we validate that Libra's default $ratio_{global}$ (i.e., 50%) has fully explored the potential of tolerance-aware compression. As shown in Table 4, further compressing KV cache to a more aggressive global ratio of 35% incurs unignorable quality loss of 4.6%/2.8%. Although a 35% ratio is acceptable on several datasets such as Xsum or Samsum, Libra will not choose it as its default configuration since Libra serves as a general system service. Notably, in this case Libra still achieves higher average accuracy compared to on-par static methods, providing a competitive accuracy-latency/memory trade-off for OS.

Comparison to SoTA KV compression methods. In Table 5, we report the maximal compression ratio (obtained at a step size of 5%) to FP16 when preserving MMLU accuracy. Libra achieves a better compression ratio compared to sparsification (H_2O) and prompt compression (LLMLingua2) by exploring a unique dynamic quantization design space. Notably, Libra is compatible with the above two methods in philosophy, and our experiments ("+H_2O" and "+LG2" columns) show that Libra achieves even better compression ratio to FP16 on top of them.

SoTA quantization-based method CacheGen outperforms Libra by leveraging quantization together with Arithmetic Coding (AC) [81]. However, the encoding/decoding of AC is extremely costly on mobile devices. We report time overhead of compressing/decompressing 128 tokens of Llama2-7B on MI14 in Table 5. It takes 47.9s for CacheGen to perform AC, being unacceptable for LLMaaS. In comparison, Libra is friendly to LLMaaS cause it only performs a lightweight linear quantization.

Discussion on choosing quantization as Libra's default compression method. As shown in the design section, Libra employs linear quantization as the default compression strategy. Quantization is the most well-studied way for DNN compression, with substantial literature showing its effectiveness [39, 41, 71]. Apart from quantization, there are also KV cache compression methods such as sparsification, coding based methods and learning based methods. As discussed before in Table 5, quantization is orthogonal to sparsification (e.g., H_2O); coding based methods (e.g., CG) requires stronger compute ability, which is constrained on mobile devices; learning based methods (e.g., Multi-Head Latent Attention) cannot be plug-and-play for a vanilla LLM architecture. In comparison, quantization offers the most easy and lightweight way for multi-level KV cache compression. Besides, with quantization, the possible compression level of each chunk is strictly limited (e.g., 2bits/4bits/8bits), making it easier for global planning.

5.3 Ablation Study

Ablation study. We further conduct a breakdown analysis of the benefit brought by Libra's each technique. The experiments are performed on Jetson Orin NX. The results are illustrated in Figure 12. We observe that all techniques have non-trivial contribution to the improvement. For instance, when using Llama2 model to serve 8 active contexts that called in a Markov pattern, Libra takes 0.27 seconds to switch to a new context on average. Without our chunk lifecycle management, this number becomes 0.62 seconds; without our tolerance-aware compression or swapping-recompute pipeline, this number becomes 0.42 seconds and 1.62 seconds, respectively.

Model	Llama2		OPT	
	A-S	R-S	A-S	R-S
Jaccard Similarity	0.77	0.23	0.82	0.16

Table 6: The effectiveness of attention scores. We report the Jaccard Similarity of top 100 tokens on 500 entries of WikiText-2 dataset. A-S means comparing attention score based importance to semantic importance. R-S means comparing random importance to semantic importance.

	Orin	TX2	MI14
Rec. config.	16–64	8–128	8–256

Table 7: Recommended chunk size configuration across various hardware. Model: Llama2; active contexts: 8.

Effectiveness of attention scores. We justify the effectiveness of attention scores by the following experiment. We rank the semantic importance of tokens by a fine-tuned small language model that proposed by LLMLingua2 [62]. As shown in Table 6, the attention score exhibits clear relevance to semantic importance, justifying its effectiveness.

5.4 Chunk Size Selection

Libra manages KV cache in chunks. In our all experiments, the chunk size is set to 16 tokens. Here we evaluate the influence of chunk size on context switching to validate this setting. In Figure 13, we report the context switching latency under various token numbers in a chunk. We observe that a too large or too small chunk size incurs undesirable switching latency. The reasons are two fold: small chunks cannot fully utilize disk bandwidth; large chunks result in redundant swapping. Therefore, Libra selects the optimal trade-off point to fully utilize the idea of chunking.

We show the generalization across various devices in Table 7. We do device-specific profiling and report the recommended configuration on each device. The results indicate that a chunk size of 16 shows strong generalization, as the tradeoff on mobile devices shows transferability.

5.5 Service Stability Analysis

We analyze Libra's influence on the stability of LLM Service.

Influence on LLM inference. By design, Libra aims to minimize the context switching latency. In Figure 14a, we compare the performance of LLM inference between scenarios with and without the use of Libra. As observed, there is no significant performance difference (within 5%).

Sensitivity to service calling frequency. Recall that we generate the trace's LLMaaS calling time with a Poisson distribution, which is influenced by the calling rate, i.e., service calling frequency. In Figure 14b, we report the switching latency under various request interval with 16 active contexts on Jetson Orin NX. Libra demonstrates stable context switching latency at both high and low calling frequencies.

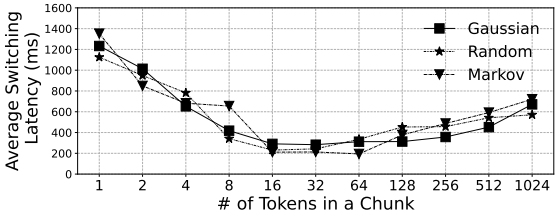
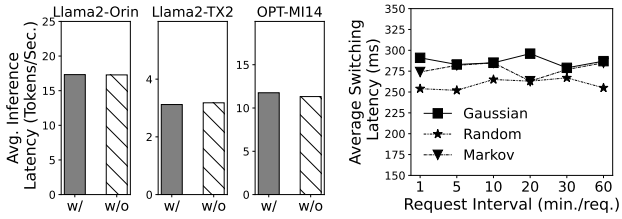


Figure 13: Influence of chunk size. Device: Orin; model: Llama2; active contexts: 8.



(a) Influence on LLM inference performance. **(b) Sensitivity to service calling frequency.**

Figure 14: Libra's influence on LLMaaS stability. "w/" and "w/o" in (a) means managing context memory with and without Libra, respectively.

6 Related Work

Foundation models on devices. AI has become a prevalent workload on mobile devices [67, 68]. Empowered by the recent progress, especially LLMs in ML community, some work proposes replacing fragmented task-specific models with a one-size-fits-all foundation model [51, 72, 74, 83, 90, 93, 96]. Such models have larger parameter sizes, stronger general-purpose task capabilities, and support multiple modalities. For instance, M4 [96] achieves comparable accuracy to specialized models on five modalities (image/text/audio/IMU/mix) with a multi-path executed architecture. With the support of corresponding software [10, 59, 70, 73, 87] and hardware [19], a plethora of revolutionary mobile applications [4, 12, 16, 30, 78, 86, 89] are built based on foundation models. One killer app among them is the LLM agent-based UI automation [49, 78]. For instance, Autodriod [78] employs Vicuna [32] to complete an arbitrary task by interacting with the smartphone GUI. Libra sheds light on deploying the aforementioned foundation models on devices. At the OS level, Libra provides opportunities to leverage NPU or batching; at the application level, by treating context as LLMaaS interface, Libra can provide personalized and stateful services for apps.

Swapping-based mobile OS memory management. A considerable amount of work [40, 45, 47, 103, 104] employs swapping to mitigate the cold-start latency caused by low-memory killer. For instance, MARS [40] optimizes Linux swapping to enhance performance on flash storage devices. By deactivating garbage collection, it reclaims memory from background apps. Exploiting the opportunity of KV cache, Libra's swapping differs from these approaches. For instance, its swapping operates at the granularity of token chunks, rather than pages or objects. Also, in contrast to lossless app memory compression (e.g., zram [20]), Libra introduces approximations for chunks.

KV cache approximation. KV cache facilitates LLM in memorizing historical knowledge. Some recent work [21, 54, 55, 84, 97, 100, 102] focuses on optimizing KV cache by sparsification or quantization. Dynamic sparsification methods, such as Big Bird [97], only mitigates the compute overhead; static sparsification methods, such as H_2O [100], reduces memory footprint by permanently removing tokens from subsequent LLM decoding. Regarding quantization, a considerable amount of work [14, 41, 84] can losslessly quantize KV cache to 8-bit integers. Some work [50, 101] even propose quantizing it to 4 bits, partly sacrificing generation quality for lower memory consumption. Libra's compression is orthogonal to these techniques. Atop them, it further performs more aggressive quantization on less informative chunks to achieve a better accuracy-memory trade-off.

Uniqueness to existing cloud-targeted LLM context management systems. Libra is an algorithm-system co-design that focuses on swapping-centric solutions for extremely memory-constrained devices, whose RAM cannot be scaled up like GPU-clusters' total HBM. The key insight and main focus of cloud-targeted systems (e.g., vLLM) are to minimize memory (HBM) underutilization by chunking. Since HBM is scalable, the swapping is only an additional feature, with naive designs like FCFS policy and request-level swapping. Their philosophy is fundamental yet completely insufficient for Libra. On top of chunking, our contributions/insights include a chunk-specific compression, a bubble-free adjustable chunk-grained pipeline and a careful lifecycle management of each chunk (e.g., Ahead-of-Time swapping and LCTRU-queue) that fully explore the potential of swapping.

Comparison to MemGPT. MemGPT [61] also talks about the memory management of on-device LLM. However, the main focus of MemGPT is fundamentally different from our system. MemGPT augments the LLM memory ability through storing the text as external memory and replacing the context window by the recorded historical text. At system level, it needs to recompute the whole context as no KV cache is cached. In comparison, Libra stores the KV cache as the LLM memory. Technically, the new philosophy of our work is that we manage the context memory in flexible chunk, an advanced memory segment compared to naive blocked memory that adopted by frameworks such as vLLM, as they do not have to consider the extreme low RAM issue on mobile devices.

Tiered memory/activation offloading. DNN serving/training systems typically mitigate the memory pressure by offloading the activation (e.g. KV cache) to lower tier memory like CPU DRAM or SSD. For instance, Pie [91] swaps out portions of the KV cache to CPU memory when not immediately needed, freeing up GPU memory for active KV cache usage. MIRAGE [48] further swaps the model parameters out of HBM to make room for KV cache. ATP [31] offloads the activations of DNN training to CPU DRAM for a larger batch size. Generally, Libra also performs tiered memory offloading, which swaps KV cache out to disk. However, it further employs fine-grained optimizations like chunk-wise swapping and swapping-recompute pipeline, making full use of the limited RAM of mobile devices.

7 Discussion

Libra identifies and tackles the most urgent challenge of running LLMs on devices as a stateful service: the efficient management of inference context on memory-constrained devices. Beyond this, we further discuss the other details of bringing the LLM to mobile OS, and show how the on-device LLMaaS design considers about these aspects.

Permission control. To avoid the user data potentially being used by the mobile developers or malicious actors, we envision to introduce permission control to LLM service. The OS provided by vendors is typically trustable, and the raw data from/to the apps can be protected by encryption and TEE. Each app can only visit the context that completely owned by it, and needs to apply to the OS for the permission to contexts that participated by other apps. LLM service itself is also robust to attack. Its action is born to be relatively clear: it only performs LLM inference (matrix multiplication) with simple, deterministic control flow, isolated in a carefully designed permission sandbox.

Personalization. As a single instance service, LLM service needs to tackle the problem of providing personalized service to various requests. To this end, we envision to design two levels of personalization methods. For few-shot tasks, we attach the samples and instructions to the prompt context, i.e., in-context learning. For multiple-shot tasks, we on-device train lightweight LoRAs and dynamically assign them to proper requests.

Updating. As a system service, the LLM runs like a firmware, which is provided by the OS vendor and pre-embedded into the system. As a result, how to update the LLM service during the following versions is a problem. To this end, we plan to provide online updating mechanism, allowing the OS vendor to distribute the new version of LLMs through internet.

Contention of multiple apps. Currently, the design of Libra does not consider the contention of multiple apps. The main reason is that the mobile apps request the LLM service in a relatively low frequency compared to cloud system (typically batch size is 1) [29, 88]. When contention happens in extreme circumstance, as discussed before, the context involved in inference is pinned in memory as a working set. Thus, the correctness will not be influenced by contention. The latency of starvation exists due to contention. This overhead cannot be avoided, yet can be minimized by batching the requests in a window of time interval. In this case, the LCTRU queue will still mark the recent access time of each chunk. The recompute delay will be re-calibrated with various batch sizes. In general, the LCTRU queue and swapping-recompute pipeline can still work in the case of contention, without the need of significant modification.

Context budget overflow. Currently, Libra allocates the memory for each context with a fixed length, without dynamic runtime expansion/shrinking. This guarantees easy planning considering the adaptive compression and pipelining. To handle the accident overflow, Libra can over-allocate a portion of memory (e.g., 10% more tokens) for each context. The performance will be degraded; yet the degradation is acceptable since the extra memory is relatively small. We briefly analysis the overhead according to the experiments in Figure 9. With a 10% extra memory buffer, if we have 12 active contexts, the switching overhead will just fallback to 13 or 14 active contexts on various models and devices, being the same level.

8 Conclusion

This work advocates LLM as a service on mobile devices (LLMaaS), a new paradigm to fully unleash the power of on-device LLM. We then present an end-to-end LLMaaS context management system named Libra. Libra enables low-overhead LLM context switching under tight memory constraint through fine-grained, chunk-wise, globally-optimized KV cache compression and swapping. Extensive experiments demonstrate the efficacy of Libra.

Acknowledgments

This work was supported by National Natural Science Foundation of China under the grant number 62325201. Mengwei Xu is supported by NSFC (No. 62522202) and Beijing Natural Science Foundation (No. L253005).

References

[1] 2024. AICore. https://developer.android.com/ml/aicore.
[2] 2024. Andriod low-memory killer. https://developer.android.com/topic/performance/memory-management#low-memory_killer.
[3] 2024. Gboard Smart Reply. https://developers.google.com/ml-kit/language/smart-reply.
[4] 2024. Glarity. https://glarity.app/.
[5] 2024. GPT-based email writer. https://hix.ai/ai-email-writer-email-generator.
[6] 2024. GPT4-Turbo. https://platform.openai.com/docs/models/gpt-4-and-gpt-4-turbo.
[7] 2024. Jetson Orin NX. https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/jetson-orin/.
[8] 2024. Jetson TX2. https://developer.nvidia.com/embedded/jetson-tx2.
[9] 2024. Large Language Models On-Device with MediaPipe and TensorFlow Lite. https://developers.googleblog.com/2024/03/running-large-language-models-on-device-with-mediapipe-andtensorflow-lite.html.
[10] 2024. Llama.cpp. https://github.com/ggerganov/llama.cpp.
[11] 2024. LLM-based AI-Assistant. https://github.com/avsrma/LLM-based-AI-Assistant.
[12] 2024. LLM customer service and support. https://www.databricks.com/solutions/accelerators/llms-customer-service-and-support.
[13] 2024. LLM telegram chatbot. https://github.com/Fatal3xcept10n/LLM-Telegram-Chatbot.
[14] 2024. LM Deploy. https://github.com/InternLM/lmdeploy/tree/main.
[15] 2024. MI14 smartphone. https://en.wikipedia.org/wiki/Xiaomi_14.
[16] 2024. News Summarization with LLM. https://github.com/KillerStrike17/News-Summarization-with-LLM.
[17] 2024. Pickle. https://docs.python.org/3/library/pickle.html.
[18] 2024. Pickle-in-Cpp. https://github.com/Usama-Azad/Pickle-in-Cpp.
[19] 2024. Snapdragon 8 gen 3 mobile platform product brief. https://docs.qualcomm.com/bundle/publicresource/87-71408-1_REV_C_Snapdragon_8_gen_3_Mobile_Platform_Product_Brief.pdf.
[20] 2024. zRAM. https://en.wikipedia.org/wiki/Zram.
[21] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. 2024. APIServe: Efficient API Support for Large-Language Model Inferencing. arXiv:2402.01869 [cs.LG]
[22] OpenAI: Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, et al. 2023. GPT-4 Technical Report. arXiv:2303.08774 [cs.CL]
[23] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. arXiv:2312.11514 [cs.CL]
[24] Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. 2023. The Falcon Series of Open Language Models. arXiv:2311.16867 [cs.CL]
[25] Ricardo Baeza-Yates, Di Jiang, Fabrizio Silvestri, and Beverly Harrison. 2015. Predicting The Next App That You Are Going To Use. In *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining* (Shanghai, China) *(WSDM '15)*. Association for Computing Machinery, New York, NY, USA, 285–294. doi:10.1145/2684822.2685302

[26] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2016. Neural Machine Translation by Jointly Learning to Align and Translate. arXiv:1409.0473 [cs.CL]

[27] Ond rej Bojar, Rajen Chatterjee, Christian Federmann, Yvette Graham, Barry Haddow, Shujian Huang, Matthias Huck, Philipp Koehn, Qun Liu, Varvara Logacheva, Christof Monz, Matteo Negri, Matt Post, Raphael Rubino, Lucia Specia, and Marco Turchi. 2017. Findings of the 2017 Conference on Machine Translation (WMT17). In *Proceedings of the Second Conference on Machine Translation, Volume 2: Shared Task Papers*. Association for Computational Linguistics, Copenhagen, Denmark, 169–214. http://www.aclweb.org/anthology/W17-4717

[28] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. 2020. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs.CL]

[29] Le Chen, Dahu Feng, Erhu Feng, Yingrui Wang, Rong Zhao, Yubin Xia, Pinjie Xu, and Haibo Chen. 2025. Characterizing Mobile SoC for Accelerating Heterogeneous LLM Inference. arXiv:2501.14794 [cs.DC] https://arxiv.org/abs/2501.14794

[30] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior Sequence Transformer for E-commerce Recommendation in Alibaba. arXiv:1905.06874 [cs.IR]

[31] Weiduo Chen, Xiaoshe Dong, Fan Zhang, Bowen Li, Yufei Wang, and Qiang Wang. 2025. ATP: Achieving Throughput Peak for DNN Training via Smart GPU Memory Management. *ACM Trans. Archit. Code Optim.* 22, 1, Article 21 (March 2025), 27 pages. doi:10.1145/3701996

[32] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. https://lmsys.org/blog/2023-03-30-vicuna/

[33] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs.CL]

[34] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Zhiyong Wu, Baobao Chang, Xu Sun, Jingjing Xu, Lei Li, and Zhifang Sui. 2023. A Survey on In-context Learning. arXiv:2301.00234 [cs.CL]

[35] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323 [cs.LG]

[36] Thomas Mesnard Gemma Team, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, and et al. 2024. Gemma. (2024). doi:10.34740/KAGGLE/M/3301

[37] In Gim, Guojun Chen, Seung seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2023. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. arXiv:2311.04934 [cs.CL]

[38] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. 2019. SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*. Association for Computational Linguistics, Hong Kong, China, 70–79. doi:10.18653/v1/D19-5409

[39] Liwei Guo, Wonkyo Choe, and Felix Xiaozhu Lin. 2023. STI: Turbocharge NLP Inference at the Edge via Elastic Pipelining. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '23)*. ACM, 791–803. doi:10.1145/3575693.3575698

[40] Weichao Guo, Kang Chen, Huan Feng, Yongwei Wu, Rui Zhang, and Weimin Zheng. 2016. *MARS* : Mobile Application Relaunching Speed-Up through Flash-Aware Page Swapping. *IEEE Trans. Comput.* 65, 3 (2016), 916–928. doi:10.1109/TC.2015.2428692

[41] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. arXiv:1510.00149 [cs.CV]

[42] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. arXiv:2009.03300 [cs.CY]

[43] Karl Moritz Hermann, Tomás Kociský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. 2015. Teaching Machines to Read and Comprehend. In *NIPS*. 1693–1701. http://papers.nips.cc/paper/5945-teaching-machines-to-read-and-comprehend

[44] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL]

[45] Sang-Hoon Kim, Jinkyu Jeong, and Jin-Soo Kim. 2017. Application-Aware Swapping for Mobile Systems. *ACM Trans. Embed. Comput. Syst.* 16, 5s, Article 182 (sep 2017), 19 pages. doi:10.1145/3126509

[46] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

[47] Niel Lebeck, Arvind Krishnamurthy, Henry M. Levy, and Irene Zhang. 2020. End the Senseless Killing: Improving Memory Management for Mobile Operating Systems. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 873–887. https://www.usenix.org/conference/atc20/presentation/lebeck

[48] Ruihao Li, Shagnik Pal, Vineeth Narayan Pullu, Prasoon Sinha, Jeeho Ryoo, Lizy K. John, and Neeraja J. Yadwadkar. 2025. MIRAGE: KV Cache Optimization through Parameter Remapping for Multi-tenant LLM Serving. arXiv:2507.11507 [cs.OS] https://arxiv.org/abs/2507.11507

[49] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanjing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. *arXiv preprint arXiv:2401.05459* (2024).

[50] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024. QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving. arXiv:2405.04532

[51] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. arXiv:2304.08485 [cs.CV]

[52] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Language Model Serving. arXiv:2310.07240

[53] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. 2024. The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits. arXiv:2402.17764 [cs.CL]

[54] Zehong Ma, Longhui Wei, Feng Wang, Shiliang Zhang, and Qi Tian. 2025. MagCache: Fast Video Generation with Magnitude-Aware Cache. arXiv:2506.09045 [cs.CV] https://arxiv.org/abs/2506.09045

[55] Zehong Ma, Shiliang Zhang, Longhui Wei, and Qi Tian. 2025. Efficient Multimodal Long Context Learning for Training-free Adaptation. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=6Rvs8jluQP

[56] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. 2022. PEFT: State-of-the-art Parameter-Efficient Fine-Tuning methods. https://github.com/huggingface/peft.

[57] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer Sentinel Mixture Models. arXiv:1609.07843 [cs.CL]

[58] Sewon Min, Xinxi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? arXiv:2202.12837 [cs.CL]

[59] mllm team. 2023. *mllm*. https://github.com/UbiquitousLearning/mllm

[60] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. 2018. Don't Give Me the Details, Just the Summary! Topic-Aware Convolutional Neural Networks for Extreme Summarization. *ArXiv* abs/1808.08745 (2018).

[61] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs.AI] https://arxiv.org/abs/2310.08560

[62] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. arXiv:2403.12968

[63] Abhinav Parate, Matthias Böhmer, David Chu, Deepak Ganesan, and Benjamin M. Marlin. 2013. Practical prediction and prefetch for faster access to applications on mobile phones. In *Proceedings of the 2013 ACM International Joint Conference on Pervasive and Ubiquitous Computing* (Zurich, Switzerland) *(UbiComp '13)*. Association for Computing Machinery, New York, NY, USA, 275–284. doi:10.1145/2493432.2493490

[64] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2022. Efficiently Scaling Transformer Inference. arXiv:2211.05102 [cs.LG]

[65] Bozidar Radunovic and Jean-Yves Le Boudec. 2007. A Unified Framework for Max-Min and Min-Max Fairness With Applications. *IEEE/ACM Transactions on Networking* 15, 5 (2007), 1073–1083. doi:10.1109/TNET.2007.896231

[66] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ Questions for Machine Comprehension of Text. arXiv:1606.05250 [cs.CL]

[67] Leming Shen, Qiang Yang, Xinyu Huang, Zijing Ma, and Yuanqing Zheng. 2025. GPIoT: Tailoring Small Language Models for IoT Program Synthesis and Development. arXiv:2503.00686 [cs.SE] https://arxiv.org/abs/2503.00686

[68] Leming Shen, Qiang Yang, Yuanqing Zheng, and Mo Li. 2025. AutoIOT: LLM-Driven Automated Natural Language Programming for AIoT Applications. arXiv:2503.05346 [cs.CL] https://arxiv.org/abs/2503.05346

[69] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, David Yarowsky, Timothy Baldwin, Anna Korhonen, Karen Livescu, and Steven

Bethard (Eds.). Association for Computational Linguistics, Seattle, Washington, USA, 1631–1642. https://aclanthology.org/D13-1170
[70] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. 2023. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. arXiv:2312.12456 [cs.LG]
[71] Yi Su, Yuechi Zhou, Quantong Qiu, Juntao Li, Qingrong Xia, Ping Li, Xinyu Duan, Zhefeng Wang, and Min Zhang. 2025. Accurate KV Cache Quantization with Outlier Tokens Tracing. arXiv:2505.10938 [cs.CL] https://arxiv.org/abs/2505.10938
[72] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, et al. 2023. Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805 [cs.CL]
[73] MLC team. 2023. *MLC-LLM.* https://github.com/mlc-ai/mlc-llm
[74] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]
[75] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention Is All You Need. arXiv:1706.03762 [cs.CL]
[76] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling Conversational Interaction with Mobile UI using Large Language Models. arXiv:2209.08655 [cs.HC]
[77] Bryan Wang, Gang Li, and Yang Li. 2023. Enabling Conversational Interaction with Mobile UI using Large Language Models *(CHI '23)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3544548.3580895
[78] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2023. Empowering llm to use smartphone for intelligent task automation. *arXiv preprint arXiv:2308.15272* (2023).
[79] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. AutoDroid: LLM-powered Task Automation in Android. arXiv:2308.15272
[80] Hao Wen, Hongming Wang, Jiaxuan Liu, and Yuanchun Li. 2024. DroidBot-GPT: GPT-powered UI Automation for Android. arXiv:2304.07061 [cs.SE]
[81] Ian H. Witten, Radford M. Neal, and John G. Cleary. 1987. Arithmetic coding for data compression. *Commun. ACM* 30, 6 (jun 1987), 520–540. doi:10.1145/214762.214771
[82] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, Online, 38–45. https://www.aclweb.org/anthology/2020.emnlp-demos.6
[83] Shengqiong Wu, Hao Fei, Leigang Qu, Wei Ji, and Tat-Seng Chua. 2023. NExT-GPT: Any-to-Any Multimodal LLM. arXiv:2309.05519 [cs.AI]
[84] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438 [cs.CL]
[85] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient Streaming Language Models with Attention Sinks. *arXiv* (2023).
[86] Le Xiao and Xiaolin Chen. 2023. Enhancing LLM with Evolutionary Fine Tuning for News Summary Generation. arXiv:2307.02839 [cs.CL]
[87] Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2023. LLMCad: Fast and Scalable On-device Large Language Model Inference. arXiv:2309.04255 [cs.NI]
[88] Daliang Xu, Wangsong Yin, Hao Zhang, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. 2025. EdgeLLM: Fast On-Device LLM Inference With Speculative Decoding. *IEEE Transactions on Mobile Computing* 24, 4 (2025), 3256–3273. doi:10.1109/TMC.2024.3513457
[89] Huatao Xu, Liying Han, Qirui Yang, Mo Li, and Mani Srivastava. 2024. Penetrative AI: Making LLMs Comprehend the Physical World. arXiv:2310.09605 [cs.AI]
[90] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, et al. 2024. A Survey of Resource-efficient LLM and Multimodal Foundation Models. arXiv:2401.08092 [cs.LG]
[91] Yi Xu, Ziming Mao, Xiangxi Mo, Shu Liu, and Ion Stoica. 2024. Pie: Pooling CPU Memory for LLM Inference. arXiv:2411.09317 [cs.LG] https://arxiv.org/abs/2411.09317
[92] Tingxin Yan, David Chu, Deepak Ganesan, Aman Kansal, and Jie Liu. 2012. Fast app launching for mobile devices using predictive user context. In *Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services* (Low Wood Bay, Lake District, UK) *(MobiSys '12)*. Association for Computing Machinery, New York, NY, USA, 113–126. doi:10.1145/2307636.2307648
[93] Bufang Yang, Lixing He, Neiwen Ling, Zhenyu Yan, Guoliang Xing, Xian Shuai, Xiaozhe Ren, and Xin Jiang. 2023. EdgeFM: Leveraging Foundation Model for Open-set Learning on the Edge. arXiv:2311.10986 [cs.LG]
[94] Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. 2023. EdgeMoE: Fast On-Device Inference of MoE-based Large Language Models. arXiv:2308.14352 [cs.LG]
[95] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu
[96] Jinliang Yuan, Chen Yang, Dongqi Cai, Shihe Wang, Xin Yuan, Zeling Zhang, Xiang Li, Dingge Zhang, Hanzi Mei, Xianqing Jia, Shangguang Wang, and Mengwei Xu. 2023. Rethinking Mobile AI Ecosystem in the LLM Era. arXiv:2308.14363 [cs.AI]
[97] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2021. Big Bird: Transformers for Longer Sequences. arXiv:2007.14062 [cs.LG]
[98] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. OPT: Open Pre-trained Transformer Language Models. arXiv:2205.01068 [cs.CL]
[99] Xiang Zhang, Junbo Jake Zhao, and Yann LeCun. 2015. Character-level Convolutional Networks for Text Classification. In *NIPS*.
[100] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. arXiv:2306.14048 [cs.LG]
[101] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2023. Atom: Low-bit Quantization for Efficient and Accurate LLM Serving. arXiv:2310.19102 [cs.LG]
[102] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. Efficiently Programming Large Language Models using SGLang. arXiv:2312.07104 [cs.AI]
[103] Kan Zhong, Xiao Zhu, Tianzheng Wang, Dan Zhang, Xianlu Luo, Duo Liu, Weichen Liu, and Edwin H.-M. Sha. 2014. DR. Swap: Energy-efficient paging for smartphones. In *2014 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. 81–86. doi:10.1145/2627369.2627647
[104] Xiao Zhu, Duo Liu, Kan Zhong, Jinting Ren, and Tao Li. 2017. SmartSwap: High-performance and user experience friendly swapping in mobile systems. In *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1145/3061639.3062317